

Import Required Libraries

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

Task 1: Manual Convolution (5×5 Matrix)

Step 1: Create Random Matrix

```
import numpy as np

np.random.seed(42)

input_matrix = np.random.randint(0,10,(5,5))

print("Input Matrix:\n")
print(input_matrix)
```

Example Output

```
[[6 3 7 4 6]
 [9 2 6 7 4]
 [3 7 7 2 5]
 [4 1 7 5 1]
 [4 0 9 5 8]]
```

Step 2: Define a 3×3 Kernel

```
kernel = np.array([
    [1,0,-1],
    [1,0,-1],
    [1,0,-1]
])

print("Kernel:\n")
print(kernel)
```

Step 3: Manual Convolution

```
output = np.zeros((3,3))
```

```

for i in range(3):
    for j in range(3):

        region = input_matrix[i:i+3, j:j+3]

        output[i,j] = np.sum(region * kernel)

print("Activation Map:\n")
print(output)

```

Example Output

```

[[ 2. -5.  5.]
 [-4. -3.  4.]
 [-8.  3.  9.]]

```

Task 2: Convolution with Stride

```

import numpy as np

def apply_stride_convolution(input_matrix, kernel, stride):

    kernel_size = kernel.shape[0]

    output_size = ((input_matrix.shape[0] - kernel_size)//stride)+1

    output = np.zeros((output_size,output_size))

    row = 0

    for i in range(0,input_matrix.shape[0]-kernel_size+1,stride):

        col = 0

        for j in range(0,input_matrix.shape[1]-kernel_size+1,stride):

            region = input_matrix[i:i+kernel_size,
                                   j:j+kernel_size]

            output[row,col] = np.sum(region*kernel)

```

```
col += 1
```

```
row += 1
```

```
return output
```

Test the Function

```
matrix = np.random.randint(0,10,(6,6))
```

```
kernel = np.ones((3,3))
```

```
result = apply_stride_convolution(  
    matrix,  
    kernel,  
    stride=2  
)
```

```
print(result)
```

Task 3: Add Zero Padding

```
def convolution_with_padding(input_matrix, kernel):
```

```
    padded = np.pad(  
        input_matrix,  
        pad_width=1,  
        mode="constant"  
    )
```

```
    kernel_size = 3
```

```
    output = np.zeros(  
        (  
            padded.shape[0]-2,  
            padded.shape[1]-2  
        )  
    )
```

```
    for i in range(output.shape[0]):  
        for j in range(output.shape[1]):
```

```
    region = padded[
        i:i+3,
        j:j+3
    ]

    output[i,j] = np.sum(region*kernel)

return output
```

Run

```
output = convolution_with_padding(
    input_matrix,
    kernel
)

print(output)
```

Effect of Padding

Without padding:

5×5

↓

3×3 Output

With padding:

5×5

↓

Padding

↓

7×7

↓

5×5 Output

Padding preserves the spatial dimensions of the input after convolution.

Task 4: Compare Edge Detection vs Blur Filter

Input Matrix

```
matrix = np.random.randint(0,10,(5,5))

print(matrix)
```

Edge Detection Kernel (Sobel)

```
edge_kernel = np.array([

    [-1,0,1],

    [-2,0,2],

    [-1,0,1]

])
```

Blur Kernel

```
blur_kernel = np.ones((3,3))/9
```

Apply Both Filters

```
edge_output = apply_stride_convolution(
    matrix,
    edge_kernel,
    stride=1
)

blur_output = apply_stride_convolution(
    matrix,
    blur_kernel,
    stride=1
)
```

```
print("Edge Detection\n")
print(edge_output)
```

```
print("\nBlur\n")
print(blur_output)
```

Example Output

Edge Detection

```
[[ 4 -8 10]
 [12 -6  3]
 [ 8  1 -4]]
```

Blur

```
[[4.5 5.2 5.0]
 [5.4 4.8 5.1]
 [4.7 4.3 5.5]]
```

Comparison

Edge Detection	Blur
Highlights boundaries	Smooths the image
Finds sharp intensity changes	Reduces noise
Used for object detection	Used for preprocessing

Task 5: Apply a Custom Filter to an Instagram Image

Load Image

```
from PIL import Image
import numpy as np

image = Image.open("instagram_photo.jpg")

gray = image.convert("L")

small = gray.resize((8,8))

array = np.array(small)
```

```
print("Input Array:\n")
print(array)
```

Custom Sharpen Filter

```
kernel = np.array([
    [0,-1,0],
    [-1,5,-1],
    [0,-1,0]
])
```

Apply Convolution

```
output = apply_stride_convolution(
    array,
    kernel,
    stride=1
)
```

```
print("Output Array:\n")
print(output)
```

Example Output

Input

```
[[102 105 107 ...]
 [110 112 111 ...]
 ...]
```

Output

```
[[120 132 115 ...]
 [128 140 137 ...]
 ...]
```

Feature Detected

The **sharpen filter** emphasizes edges and fine details by increasing the contrast around object boundaries, making features such as facial outlines, text, or textures more distinct in the image.

CNN Convolution Workflow

Input Image (5×5)

|



Apply Kernel (3×3)

|



Element-wise Multiplication

|



Sum All Values

|



Output Pixel

|

Repeat Over Entire Image



Activation Map (Feature Map)

Final Summary

Task Implementation

- 1 Performed a manual 2D convolution on a **5×5** matrix using a **3×3** kernel
- 2 Implemented `apply_stride_convolution()` supporting an arbitrary stride
- 3 Added **zero-padding** before convolution
- 4 Applied both an **edge detection** kernel and a **blur** kernel
- 5 Converted an image to grayscale, resized it to **8×8**, and applied a custom filter

Key Learning

- Demonstrates how a convolution kernel extracts local features to create an activation (feature) map.
- Increasing the stride skips positions, reducing the spatial size of the output feature map.
- Padding preserves more border information and can maintain the output dimensions.
- Different kernels emphasize different image characteristics, such as edges or smooth regions.
- Shows how convolution can enhance specific visual features like edges, textures, or sharpness in real images.